

Computer Vision Foundations

Motivation & Applications

Deep Learning has driven rapid advancements in Computer Vision, enabling new applications and cross-fertilization of research into other fields (e.g., speech recognition).

- **Self-Driving Cars:** Locating pedestrians and other vehicles for avoidance.
- **Face Recognition:** Authentication for unlocking devices or doors.
- **Content Recommendation:** Apps identifying relevant or attractive images (food, scenery).
- **Generative Art:** Creating new types of artistic imagery.

Common Computer Vision Problems

1. Image Classification (Recognition):

- Input: Image (e.g., 64×64).
- Output: Class label (e.g., "Is this a cat?").

2. Object Detection:

- Input: Image containing multiple objects.
- Output: Position and bounding boxes of specific objects (e.g., other cars) within the scene.

3. Neural Style Transfer:

- Input: A **Content Image** and a **Style Image** (e.g., a Picasso painting).
- Output: The content image repainted in the artistic style of the style image.

The Challenge of High-Resolution Images

Standard Fully Connected Neural Networks scale poorly with image size due to the massive increase in input feature dimensions.

Input Dimensions Analysis

- **Small Image (64×64 px):**
 - Dimensions: $64 \times 64 \times 3$ (RGB channels).
 - Input Feature Vector (x): 12, 288 dimensions.
 - *Result:* Manageable for standard networks.
- **Large Image (1000×1000 px):**
 - Dimensions: $1000 \times 1000 \times 3$.
 - Input Feature Vector (x): 3, 000, 000 dimensions (3 million).

Computational Infeasibility

If a standard fully connected network is used for the large image (1000×1000):

- **Layer 1 Settings:** Assume 1, 000 hidden units.
- **Weight Matrix ($W^{[1]}$) Calculation:**
Matrix Dimensions = (Hidden Units) $\times$ (Input Features)
 $1, 000 \times 3, 000, 000 = 3, 000, 000, 000$ parameters
- **Consequences of 3 Billion Parameters:**
 1. **Overfitting:** Extremely difficult to obtain enough data to prevent overfitting.
 2. **Resource Constraints:** Computation and memory requirements are infeasible for training.

Solution

To handle large images effectively, the **Convolution Operation** is required. This is the fundamental building block of **Convolutional Neural Networks (CNNs)**.

Edge Detection Example

The Convolution Operation

The convolution operation is a fundamental building block of Convolutional Neural Networks (CNNs). While early layers might detect simple features like edges, deeper layers can detect parts of objects, and eventually complete objects (e.g., faces).

Mechanism

To demonstrate how convolution works, consider a grayscale image and a filter:

- **Input:** A 6×6 grayscale image (represented as a $6 \times 6 \times 1$ matrix).
- **Filter (or Kernel):** A 3×3 matrix.
 - *Note:* In research papers, this is often called a **kernel**, but "filter" is the standard terminology for this course.
- **Operation:** The input image is **convolved** (denoted by the asterisk $*$) with the filter to produce an output matrix.

Step-by-Step Calculation

Given a 6×6 input and a 3×3 filter, the output will be a 4×4 matrix.

1. **Overlay:** Place the 3×3 filter over the top-left 3×3 region of the input image.
2. **Element-wise Product & Sum:** Multiply each element of the filter by the corresponding element of the image region underneath it, then sum all 9 resulting numbers.
 - *Example Calculation:*
 $(1 \times 3) + (1 \times 1) + (1 \times 2) + (0 \times 0) + \dots + (-1 \times -1) = -5$
This sum (e.g., -5) becomes the top-left element of the output matrix.
3. **Shift:** Move the filter one step to the right (stride of 1) and repeat the calculation to get the next element.
4. **Repeat:** Continue shifting right until the row is complete, then shift down one row and repeat until the entire image is covered.

Implementation Note

In programming languages, the asterisk $*$ usually denotes element-wise multiplication. Deep learning frameworks provide specific functions for convolution:

- **TensorFlow:** `tf.nn.conv2d`
 - **Keras:** `Conv2D`
-

Vertical Edge Detection

We can detect specific features, such as vertical edges, by carefully selecting the values within the filter.

The Vertical Edge Filter

A common 3×3 filter used to detect vertical edges looks like this:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

- **Logic:** This filter has bright pixels (1) on the left, neutral (0) in the middle, and dark pixels (-1) on the right. It responds strongly to changes in contrast from left to right.

Simplified Example

Consider a synthetic image where the left half is bright (pixel value 10) and the right half is dark (pixel value 0):

$$\text{Input Image} = \begin{bmatrix} 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \end{bmatrix}$$

When convolved with the vertical edge filter above, the output matrix is:

$$\text{Output} = \begin{bmatrix} 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \end{bmatrix}$$

Interpretation

- **The Output Values:** The high values (30) in the center of the output matrix correspond to the boundary between the 10s and the 0s in the input.
 - **Visual Representation:** If plotted as an image, the output would show a bright vertical band in the middle, effectively highlighting where the vertical edge exists in the original image.
 - **Intuition:** The convolution operation calculates the difference between the left and right pixels within the 3×3 window. If there is a sharp transition (edge), the result is a large number.
-

Deep Learning for Edge Detection

Positive vs. Negative Edges

The convolution operation distinguishes between the direction of pixel intensity transitions (light-to-dark vs. dark-to-light).

Example: Vertical Edge Detection

Using the standard vertical filter $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$:

1. Light-to-Dark Transition:

- **Input:** Bright pixels (10) on the left, dark pixels (0) on the right.
- **Output:** Positive values (e.g., +30) in the center column.

2. Dark-to-Light Transition:

- **Input:** Dark pixels (0) on the left, bright pixels (10) on the right.
 - **Output:** Negative values (e.g., -30) in the center column.
- **Note:** If the direction of the transition does not matter for a specific application, you can take the **absolute value** of the output matrix.

Horizontal Edge Detection

Just as a filter can detect vertical edges, a rotated filter can detect horizontal edges.

- **Horizontal Filter:**

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

- **Interpretation:** This filter highlights rows where the top pixels are bright and the bottom pixels are dark (positive horizontal edge) or vice versa (negative horizontal edge).

Historical Filters (Hand-Coded)

Before deep learning, researchers debated the best numbers to use for edge detection filters. Common variations include:

1. Sobel Filter:

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

- **Characteristic:** Puts more weight on the central row.
- **Benefit:** Increases robustness (denoising).

2. Scharr Filter:

$$\begin{bmatrix} 3 & 0 & -3 \\ 10 & 0 & -10 \\ 3 & 0 & -3 \end{bmatrix}$$

- **Characteristic:** Uses significantly higher weights.
- **Benefit:** Captures different statistical properties of the image.

Learning Filters via Backpropagation

The most powerful idea in modern computer vision is to **not** hand-code these filter values.

- **Concept:** Treat the 9 numbers in the 3×3 filter as **parameters** (weights) to be learned.
 - **Mechanism:**
 - Initialize the filter parameters randomly.
 - Use **Backpropagation** to adjust these numbers based on the training data.
 - **Advantages:**
 1. **Flexibility:** The network can learn to detect vertical edges, horizontal edges, or edges at any angle (e.g., 45° , 70°).
 2. **Robustness:** The network learns features that are statistically best for the specific dataset, often outperforming human-designed filters (Sobel, Scharr, etc.).
 3. **Generalization:** It can learn complex low-level features that may not have a simple English name or definition but are critical for the task.
-

Padding in Convolutional Neural Networks

Motivation: Why Pad?

When applying standard convolutions (without padding), two significant problems arise:

1. Shrinking Output:

- With an $n \times n$ image and an $f \times f$ filter, the output dimension is $(n - f + 1) \times (n - f + 1)$.
- *Example:* A 6×6 image convolved with a 3×3 filter results in a 4×4 output.
- **The Issue:** In deep networks with many layers, the image shrinks rapidly, potentially becoming too small (1×1) before sufficient feature extraction occurs.

2. Information Loss at Edges:

- Pixels in the corners or edges of an image are used in fewer convolution steps compared to pixels in the center.
- *Visual:* A corner pixel is only "touched" by the filter once, whereas a center pixel is overlapped by many different filter positions.
- **The Issue:** Much of the information near the borders of the image is effectively thrown away or under-utilized.

The Solution: Padding

To resolve these issues, we can pad the input image with an additional border of pixels (usually zeros) before convolving.

- **Notation:** Let p be the padding amount (number of pixel layers added to the border).
- **New Dimensions:**
 - Input changes from $n \times n$ to $(n + 2p) \times (n + 2p)$.
 - Output dimension becomes:
 $(n + 2p - f + 1) \times (n + 2p - f + 1)$
- **Example:**
 - Input: 6×6 ($n = 6$)
 - Filter: 3×3 ($f = 3$)

- Padding: $p = 1$
- Resulting Output: $(6 + 2(1) - 3 + 1) = 6$. The output is 6×6 , preserving the original size.

Types of Convolutions

There are two common choices for padding, often specified as arguments in deep learning frameworks:

1. Valid Convolution

- **Definition:** No padding is used ($p = 0$).
- **Output Size:** $(n - f + 1) \times (n - f + 1)$.
- **Effect:** The image shrinks with every layer.

2. Same Convolution

- **Definition:** Pad so that the output size equals the input size.
- **Formula:**
To satisfy $n + 2p - f + 1 = n$, we solve for p :
$$p = \frac{f-1}{2}$$
- **Example:**
 - If $f = 3$, then $p = (3 - 1)/2 = 1$.
 - If $f = 5$, then $p = (5 - 1)/2 = 2$.

Convention: Odd-Numbered Filters

In computer vision, filters ($f \times f$) are almost always **odd** (e.g., 3×3 , 5×5 , 7×7).

- **Reason 1 (Symmetry):** If f is even, you would need asymmetric padding (e.g., pad 1 on left, 2 on right) to maintain dimensions. Odd filters allow for symmetric padding ($p = (f - 1)/2$).
 - **Reason 2 (Central Pixel):** Odd filters have a distinct central pixel, which helps in defining the "position" of the filter relative to the input.
-

Strided Convolutions

Concept

Strided Convolution is a modification to the basic convolution operation where the filter "steps" or "hops" over the input by a specified number of pixels, rather than shifting one pixel at a time.

- **Stride (s):** The number of pixels the filter shifts at each step.
- **Mechanism:**
 1. Perform the element-wise product and sum at the starting position.
 2. Shift the filter by s pixels to the right (instead of 1).
 3. Repeat until the row is complete.
 4. Shift the filter down by s pixels and repeat for subsequent rows.

Output Dimensions

The dimensions of the output matrix are governed by the input size (n), filter size (f), padding (p), and stride (s).

The Formula

For an $n \times n$ image convolved with an $f \times f$ filter, padding p , and stride s :

$$\text{Output Size} = \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

- $\lfloor z \rfloor$ (**Floor Function**): This notation means rounding down to the nearest integer.
- **Constraint:** If the fraction $\frac{n+2p-f}{s}$ is not an integer, we round down.
 - *Interpretation:* A convolution computation is only performed if the filter lies **entirely** within the image (or padded image). If a filter hangs partially outside, that computation is skipped.

Example Calculation

- **Input:** 7×7 image ($n = 7$)
- **Filter:** 3×3 ($f = 3$)
- **Padding:** None ($p = 0$)
- **Stride:** 2 ($s = 2$)

$$\text{Dimension} = \frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$$

- *Result:* A 3×3 output matrix.

Technical Note: Convolution vs. Cross-Correlation

There is a technical distinction between the mathematical definition of convolution and how it is implemented in Deep Learning.

1. Mathematical Convolution:

- Requires **mirroring** (flipping) the filter horizontally and vertically before applying the element-wise product.
- *Property:* This ensures associativity: $(A * B) * C = A * (B * C)$.

2. Deep Learning "Convolution":

- **Skips the mirroring step.** We simply slide the filter over the input.
 - *Technical Name:* Strictly speaking, this operation is called **Cross-Correlation**.
 - *Convention:* In Deep Learning literature and frameworks, this is universally referred to as **Convolution**, despite the mathematical inaccuracy.
 - *Reasoning:* The mirroring step adds complexity without benefiting neural network performance. The network can simply learn the "flipped" values in the weights if necessary.
-

Convolutions Over Volumes

Convolving on RGB Images

Previously, we dealt with 2D grayscale images (6×6). Real-world applications often use **RGB images**, which have three color channels (Red, Green, Blue).

Dimensions

- **Input Image:** Instead of a flat matrix, the input is a volume.
 - Dimensions: $6 \times 6 \times 3$
 - (Height $\times$ Width $\times$ Channels)
- **The Filter:** The filter must also be 3D to match the input depth.
 - Dimensions: $3 \times 3 \times 3$
 - **Crucial Rule:** The number of channels in the filter (last dimension) **must match** the number of channels in the input image.

The Operation

1. **Overlay:** Place the $3 \times 3 \times 3$ filter (which contains 27 parameters/numbers) over the top-left corner of the input image.
2. **Calculation:**
 - Multiply each of the 27 numbers in the filter by the corresponding 27 numbers in the red, green, and blue channels of the image.
 - Sum all products together.
 - The result is a **single number** in the output matrix.
3. **Slide:** Slide the filter over (stride) and repeat.
4. **Result:** A $6 \times 6 \times 3$ image convolved with a $3 \times 3 \times 3$ filter results in a 4×4 **2D matrix** (not 3D).

Feature Detection

You can design filters to detect features in specific channels:

- **Detect Vertical Edges (Red Only):** Set the "Red" slice of the filter to the vertical edge pattern (1, 0, -1) and set the Green and Blue slices to all zeros.
 - **Detect Vertical Edges (Any Color):** Set the vertical edge pattern in all three slices (Red, Green, and Blue).
-

Using Multiple Filters

In a real neural network, you usually want to detect many features simultaneously (e.g., vertical edges, horizontal edges, 45-degree lines, etc.).

The Mechanism

1. **Filter 1 (Vertical):** Convolve input ($6 \times 6 \times 3$) with Filter 1 ($3 \times 3 \times 3$). Result $\rightarrow 4 \times 4$ matrix.
2. **Filter 2 (Horizontal):** Convolve the *same* input with Filter 2 ($3 \times 3 \times 3$). Result $\rightarrow$ distinct 4×4 matrix.
3. **Stacking:** Stack these output matrices together to form a volume.

Output Volume Dimensions

If you stack the outputs of 2 filters, the result is $4 \times 4 \times 2$.

General Formula:

$$n \times n \times n_c \quad * \quad f \times f \times n_c \quad \rightarrow \quad (n - f + 1) \times (n - f + 1) \times n'_c$$

- n : Input height/width

- f : Filter height/width
- n_c : Number of channels in input (and filter)
- n'_c : Number of filters used (becomes the number of channels in the output)

Terminology Note

- **Channels vs. Depth:** The third dimension (n_c) is often called "depth" in literature. However, to avoid confusion with the "depth of a neural network" (number of layers), this course uses the term **Channels**.
-

One Layer of a Convolutional Neural Network

The Architecture of a Single Layer

To construct a full layer in a Convolutional Neural Network (CNN), we extend the convolution operation by adding a bias and an activation function, similar to standard neural networks.

The Process

1. **Convolution:** Take the input volume (e.g., $6 \times 6 \times 3$) and convolve it with a specific filter (e.g., Filter 1).
 - Result: A 2D matrix (e.g., 4×4).
2. **Add Bias:** Add a single real number (bias, b_1) to every element of the resulting matrix.
 - *Python Implementation:* This is handled via **broadcasting**, where the scalar is added to all 16 elements of the 4×4 matrix.
3. **Non-Linearity:** Apply an activation function (e.g., ReLU) element-wise to the matrix.
 - Result: A 4×4 activated matrix.
4. **Repeat & Stack:** Repeat steps 1-3 for every filter in the layer (e.g., Filter 2). Stack the resulting matrices to form the output volume.

Comparison with Standard Neural Networks

We can map CNN operations to the standard propagation equations:
 $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$ and $A^{[l]} = g(Z^{[l]})$.

Standard NN Concept	CNN Equivalent
Input ($A^{[0]}$ or X)	The input volume (e.g., $6 \times 6 \times 3$).
Weights (W)	The filters (e.g., two $3 \times 3 \times 3$ filters).
Linear Op ($W \cdot A$)	The convolution operation.
Bias (b)	The bias scalar added to the convolution output.
Linear Output (Z)	The intermediate 4×4 matrices (before activation).
Activation ($A^{[1]}$)	The final stacked volume (e.g., $4 \times 4 \times 2$) after applying ReLU.

Parameter Counting & Efficiency

A significant advantage of CNNs is parameter sharing, which keeps the parameter count low regardless of input image size.

Example Calculation

Scenario:

- **Input:** 1000×1000 image (RGB).
- **Layer:** 10 filters, each of size $3 \times 3 \times 3$.

Calculation:

1. Parameters per Filter:

- Weights: $3 \times 3 \times 3 = 27$ parameters.
- Bias: 1 parameter.
- Total per filter: $27 + 1 = 28$.

2. Total Parameters for Layer:

- $28 \text{ parameters} \times 10 \text{ filters} = \mathbf{280 \text{ parameters}}$.

Key Insight: Even if the input image is massive (5000×5000), the number of parameters remains **280**. This drastic reduction compared to fully connected layers makes CNNs less prone to overfitting and computationally efficient.

Summary of Notation (Layer l)

Standardizing notation is critical for defining dimensions in deep CNNs.

Hyperparameters

- $f^{[l]}$: Filter size (height/width).
- $p^{[l]}$: Padding.
- $s^{[l]}$: Stride.
- $n_C^{[l]}$: Number of filters (determines output depth).

Input Dimensions (from Layer $l - 1$)

- Input Volume: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_C^{[l-1]}$
 - n_H : Height
 - n_W : Width
 - n_C : Number of channels (depth)

Filter Dimensions

- Each filter: $f^{[l]} \times f^{[l]} \times n_C^{[l-1]}$
 - Note: The filter's channels **must match** the input's channels ($n_C^{[l-1]}$).
- Weights ($W^{[l]}$): $f^{[l]} \times f^{[l]} \times n_C^{[l-1]} \times n_C^{[l]}$
 - (Filter Height $\times$ Filter Width $\times$ Input Channels $\times$ Number of Filters)
- Bias ($b^{[l]}$): Vector of size $1 \times 1 \times 1 \times n_C^{[l]}$ (one bias per filter).

Output Dimensions (Activation $A^{[l]}$)

- Output Volume: $n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$
- Height/Width Formula:
$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$
$$n_W^{[l]} = \left\lfloor \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$
- Channels: $n_C^{[l]}$ (Equal to the number of filters used in layer l).

Batch Output (Vectorized)

If processing m examples (batch gradient descent), the output dimension becomes:

$$m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$$

(Note: Some frameworks put the channel dimension first, e.g., $m \times n_C \times n_H \times n_W$. Consistency is key.)

Simple Convolutional Network Example

Concrete Architecture Walkthrough

This example demonstrates a basic deep Convolutional Neural Network (ConvNet) designed for image classification (e.g., determining if an image contains a cat).

1. Input Layer

- **Input (x):** An image of size $39 \times 39 \times 3$.
 - $n_H^{[0]} = n_W^{[0]} = 39$
 - $n_C^{[0]} = 3$ (RGB channels)

2. Layer 1 (Convolution)

- **Parameters:**
 - Filter size ($f^{[1]}$): 3×3
 - Stride ($s^{[1]}$): 1
 - Padding ($p^{[1]}$): 0 (Valid convolution)
 - Number of filters: 10
- **Output Dimension Calculation:**
$$\frac{n+2p-f}{s} + 1 = \frac{39+0-3}{1} + 1 = 37$$
- **Output Volume ($A^{[1]}$):** $37 \times 37 \times 10$
 - The depth (10) corresponds to the number of filters used.

3. Layer 2 (Convolution)

- **Parameters:**
 - Filter size ($f^{[2]}$): 5×5
 - Stride ($s^{[2]}$): 2
 - Padding ($p^{[2]}$): 0
 - Number of filters: 20
- **Output Dimension Calculation:**
$$\frac{37+0-5}{2} + 1 = \frac{32}{2} + 1 = 17$$
- **Output Volume ($A^{[2]}$):** $17 \times 17 \times 20$
 - **Observation:** The height and width shrink significantly due to the stride of 2, while the number of channels increases ($10 \rightarrow 20$).

4. Layer 3 (Convolution)

- **Parameters:**
 - Filter size ($f^{[3]}$): 5×5
 - Stride ($s^{[3]}$): 2
 - Padding ($p^{[3]}$): 0
 - Number of filters: 40
- **Output Dimension Calculation:**
$$\frac{17+0-5}{2} + 1 = \frac{12}{2} + 1 = 7$$
- **Output Volume ($A^{[3]}$):** $7 \times 7 \times 40$

5. Flattening & Output

- **Flattening:** The final volume ($7 \times 7 \times 40$) is unrolled into a single 1D vector.
 $7 \times 7 \times 40 = 1,960$ units
 - **Classification:** This vector is fed into a **Logistic Regression** or **Softmax** unit to produce the final prediction ($\hat{y}$).
-

General Trends in ConvNet Architecture

Designing a ConvNet involves selecting hyperparameters (filter size, stride, padding, number of filters). While specific choices vary, two general trends are common as you go deeper into the network:

1. **Decreasing Dimensions:** The height and width (n_H, n_W) of the image/feature map tend to decrease (e.g., $39 \rightarrow 37 \rightarrow 17 \rightarrow 7$).

2. **Increasing Channels:** The number of channels (n_C) tends to increase (e.g., $3 \rightarrow 10 \rightarrow 20 \rightarrow 40$).

Types of Layers in a ConvNet

While the example above used only Convolutional layers, a typical modern ConvNet architecture consists of three main types of layers:

1. **Convolutional Layer (CONV):** The core building block (discussed so far).
 2. **Pooling Layer (POOL):** Used to reduce dimensions (discussed next).
 3. **Fully Connected Layer (FC):** Standard dense layers used at the end of the network.
-

Pooling Layers

Purpose

Pooling layers are used alongside convolutional layers to:

1. **Reduce the size** of the representation (down-sampling).
2. **Speed up computation** by reducing dimensions.
3. Make detected features **more robust**.

Max Pooling

Max pooling is the most common type of pooling. It outputs the maximum value from a specific region of the input.

Mechanism

- **Input:** Divide the input matrix into regions based on filter size (f) and stride (s).
- **Operation:** For each region, select the **maximum number**.
- **Example:**
 - **Input:** 4×4 matrix.
 - **Hyperparameters:** $f = 2$ (filter size), $s = 2$ (stride).
 - **Output:** 2×2 matrix.
 - *Calculation:* The top-left 2×2 region of the input is examined, and the highest value becomes the top-left pixel of the output.

Intuition

- High activation numbers indicate the presence of a specific feature (e.g., a vertical edge or an eye).
- **Preservation:** If a feature exists anywhere in the filter region (quadrant), the max operation preserves that high value in the output.
- **Suppression:** If the feature does not exist (all numbers are low), the max remains small.
- *Note:* While this intuition is often cited, the primary reason for its popularity is simply that it performs well empirically.

Pooling on 3D Volumes

- Pooling applies **independently** to each channel.
- **Input:** $n_H \times n_W \times n_C$
- **Output:** $n'_H \times n'_W \times n_C$
- **Result:** The number of channels (n_C) remains **unchanged**.

Average Pooling

- **Mechanism:** Instead of taking the max, calculate the **average** of the numbers in the filter region.
- **Usage:** Much less common than max pooling in modern ConvNets.
 - *Exception:* Sometimes used very deep in the network to collapse a representation (e.g., collapsing $7 \times 7 \times 1000$ down to $1 \times 1 \times 1000$).

Hyperparameters & Parameters

Common Hyperparameters

1. **Filter Size (f):** Typically 2 or 3.
2. **Stride (s):** Typically 2.
 - *Common Setting:* $f = 2, s = 2$. This shrinks the height and width by a factor of roughly 2.
3. **Type:** Max or Average.
4. **Padding (p):** Rarely used; usually $p = 0$.

Output Dimensions

Given input size $n_H \times n_W \times n_C$:

$$\text{Output Height} = \left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor$$

$$\text{Output Width} = \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor$$

$$\text{Output Channels} = n_C$$

Learnable Parameters

- **Zero Parameters:** Pooling layers have **no learnable parameters**.
 - Gradient descent does not update anything in this layer.
 - The transformation is a fixed function defined solely by the hyperparameters (f and s).
-

CNN Architecture Example (LeNet-5 Inspired)

Overview

This segment provides a concrete walkthrough of a full Convolutional Neural Network designed for handwritten digit recognition (classifying digits 0-9). The architecture is closely inspired by the classic **LeNet-5** model created by Yann LeCun.

- **Task:** Multi-class classification (10 digits).
- **Input:** RGB Image of size $32 \times 32 \times 3$.

Layer-by-Layer Walkthrough

Layer 1 (Conv1 + Pool1)

1. Convolution (Conv1):

- **Input:** $32 \times 32 \times 3$
- **Filters:** 5×5 , Stride $s = 1$, No padding ($p = 0$).
- **Number of Filters:** 6.
- **Output:** $28 \times 28 \times 6$.
- **Calculation:** $\frac{32-5}{1} + 1 = 28$.

2. Pooling (Pool1):

- **Type:** Max Pooling.
- **Hyperparameters:** $f = 2, s = 2$.
- **Output:** $14 \times 14 \times 6$.
- **Calculation:** $\frac{28-2}{2} + 1 = 14$. The depth (6 channels) is preserved.

Layer 2 (Conv2 + Pool2)

1. Convolution (Conv2):

- **Input:** $14 \times 14 \times 6$
- **Filters:** 5×5 , Stride $s = 1$, No padding.
- **Number of Filters:** 16.
- **Output:** $10 \times 10 \times 16$.
- **Calculation:** $\frac{14-5}{1} + 1 = 10$.

2. Pooling (Pool2):

- **Hyperparameters:** $f = 2, s = 2$.
- **Output:** $5 \times 5 \times 16$.
- **Calculation:** $\frac{10-2}{2} + 1 = 5$.

Flattening

- The 3D volume from Pool2 is flattened into a 1D vector.
- **Dimensions:** $5 \times 5 \times 16 = 400$ units.

Layer 3 (Fully Connected - FC3)

- **Input:** 400 units.
- **Structure:** Standard dense neural network layer (Matrix $W^{[3]}$ + bias).
- **Units:** 120.
- **Output:** Vector of size 120.

Layer 4 (Fully Connected - FC4)

- **Input:** 120 units.
- **Units:** 84.
- **Output:** Vector of size 84.

Output Layer (Softmax)

- **Input:** 84 units.
 - **Operation:** Softmax function.
 - **Units:** 10 (representing probabilities for digits 0–9).
-

Conventions and Patterns

Layer Counting Convention

There is inconsistent terminology in the field regarding what constitutes a "layer."

- **This Course's Convention:** A layer is counted only if it has **learnable weights/parameters**.
- **Implication:** A Convolutional layer (weights) + a Pooling layer (no weights) are grouped together as **One Layer**.
 - *Example:* Conv1 + Pool1 = Layer 1.

Design Patterns

As you move deeper into a typical ConvNet:

1. **Shrinking Spatial Dimensions:** Height (n_H) and Width (n_W) decrease ($32 \rightarrow 28 \rightarrow 14 \rightarrow 10 \rightarrow 5$).
2. **Growing Depth:** The number of Channels (n_C) increases ($3 \rightarrow 6 \rightarrow 16$).

Parameter Distribution

- **Conv Layers:** Have relatively few parameters (due to parameter sharing).
- **FC Layers:** Contain the vast majority of the model's parameters.
- **Pool Layers:** Have 0 parameters.

Activation Sizes

- **Input Activation:** 3,072 units ($32 \times 32 \times 3$).
 - Activations tend to decrease gradually. A sharp/sudden drop in activation size is generally detrimental to performance.
-

Why Convolutions?

Advantages over Fully Connected Layers

There are two primary reasons why Convolutional Neural Networks (ConvNets) are superior to standard Fully Connected (FC) networks for image processing: **Parameter Sharing** and **Sparsity of Connections**.

Illustrative Example: Parameter Efficiency

Consider a transition from an input image to a convolutional layer:

- **Input:** $32 \times 32 \times 3$ image (3,072 units).
- **Conv Layer Settings:** 5×5 filters, Stride 1, 6 filters.
- **Output:** $28 \times 28 \times 6$ volume (4,704 units).

Comparison of Parameter Counts:

1. If using a Fully Connected Layer:

- Every input unit connects to every output unit.
- Weight Matrix Size: $3,072 \times 4,704 \approx$ **14 million parameters**.
- *Result:* Computationally expensive and prone to overfitting.

2. Using a Convolutional Layer:

- Each filter has $5 \times 5 = 25$ weights + 1 bias = **26 parameters**.
- With 6 filters: $26 \times 6 =$ **156 parameters**.
- *Result:* Drastic reduction in parameters.

1. Parameter Sharing

- **Definition:** A feature detector (filter) that is useful in one part of the image is likely useful in another part of the image.
- **Mechanism:** The same set of weights (e.g., a vertical edge detector) is "shared" or swept across the entire input image.
- **Benefit:** significantly reduces the number of parameters since you do not need to learn separate detectors for the upper-left and lower-right corners.

2. Sparsity of Connections

- **Definition:** Each output value depends on only a small number of input values.
- **Mechanism:** An output pixel (e.g., at position 0,0) is computed using only the

pixels within the receptive field of the filter (e.g., a 3×3 region). It is not connected to the rest of the image pixels.

- **Benefit:** reduces the computational load and enforces learning of local features.

Translation Invariance

- **Concept:** A ConvNet creates representations where a shifted object (e.g., a cat moved a few pixels to the right) results in a similar feature map.
 - **Mechanism:** Because filters are shared across the image, the network is robust to translations of objects within the scene.
-

Training a ConvNet

Putting all the building blocks together creates a trainable system for tasks like image classification.

The Pipeline

1. Architecture Design:

- Input Image x .
- Series of **Conv** and **Pool** layers.
- Flattening followed by **Fully Connected (FC)** layers.
- Output via **Softmax** ($\hat{y}$).

2. Parameters:

The network contains learnable parameters W (weights) and b (biases) in the Conv and FC layers.

3. Cost Function:

- Define a cost function J (e.g., sum of losses over the training set divided by m examples).

4. Optimization:

- Initialize parameters randomly.
- Use an optimizer (e.g., **Gradient Descent**, **RMSProp**, or **Adam**) to minimize J .